

Data Access Playground

A detailed study guide

EF Core (commands) + Dapper (queries) + Outbox/Inbox + Aspire

.NET 10 · PostgreSQL · SQL Server · RabbitMQ · Testcontainers

Contents

1. What this project is and why
2. Architecture and layers
3. Domain building blocks (DDD)
4. Strongly-typed IDs and auto-converters
5. EF Core: the command (write) side
6. Owned types: OwnsOne / OwnsMany
7. Optimistic concurrency: xmin and rowversion
8. Pessimistic locking and isolation
9. Migrations and design-time factories
0. Dapper: the query (read) side
 1. Keyset pagination
 2. CQRS and its levels
13. The transactional Outbox
14. Inbox and the idempotent consumer
15. Modern EF Core techniques
16. TimeProvider and deterministic time
17. Result vs exceptions; HTTP mapping
18. Aspire, AppHost and ServiceDefaults
19. Testing
20. EF vs Dapper benchmark
21. Conventions and code quality
22. How to run
23. Glossary

1. What this project is and why

A data-access "playground" on .NET 10 where a single domain model (orders and products) demonstrates two styles of talking to a database together with a set of production patterns.

The point is not "yet another CRUD app" but a showcase of techniques that come up in interviews and in production:

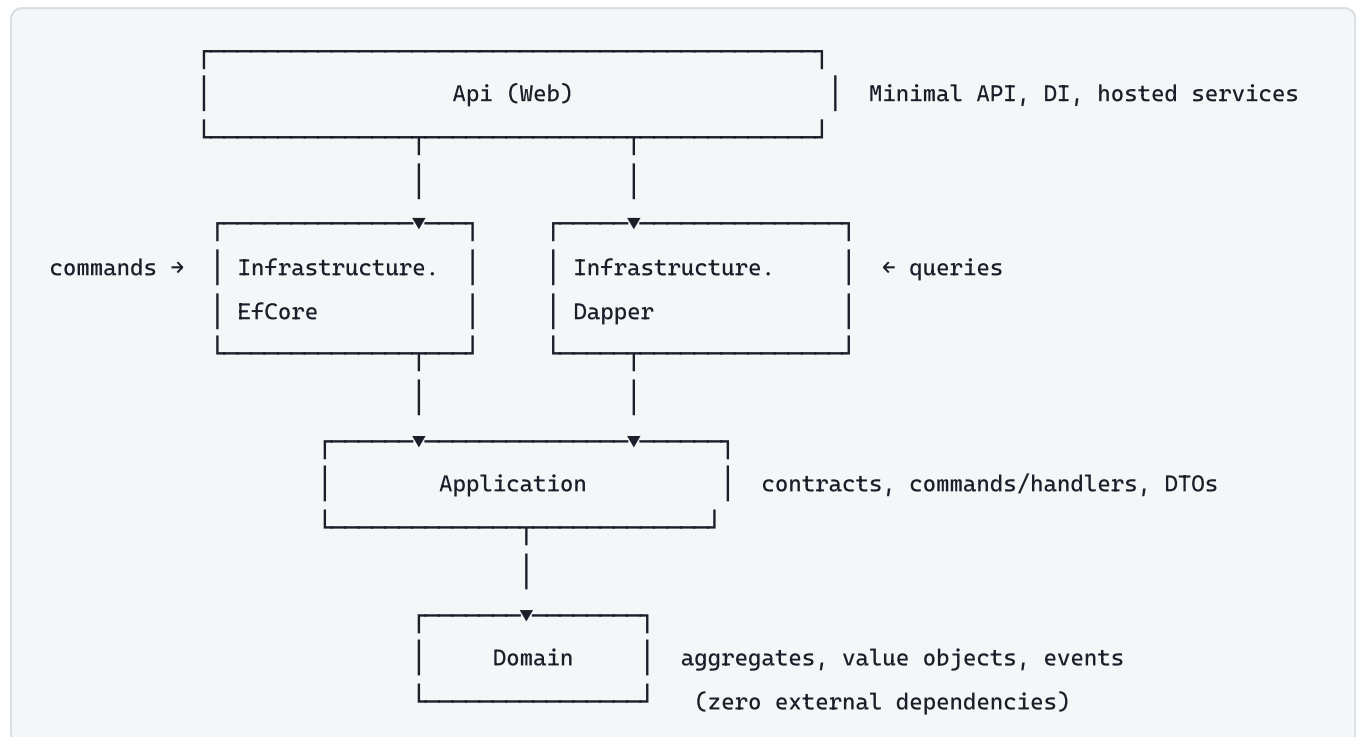
- **EF Core** — on the *command* side (state changes): aggregates, invariants, transactions, optimistic locking.
- **Dapper** — on the *query* side (reads for display): flat DTOs, hand-written SQL, multi-mapping, keyset pagination.
- **Two database providers** — PostgreSQL and SQL Server from a single codebase, with provider-specific details.
- **Transactional Outbox + Inbox** — reliable delivery of domain events to RabbitMQ.
- **.NET Aspire** — orchestrating databases and the broker for local runs.
- **Tests** — unit, a "model builds" check without a database, and integration tests on Testcontainers, plus a BenchmarkDotNet benchmark.

Stack and versions

Area	Key packages
Platform	.NET 10 (<code>net10.0</code>), C# latest, Nullable enabled, ImplicitUsings
EF Core	Microsoft.EntityFrameworkCore 10.0.4, Npgsql.EntityFrameworkCore.PostgreSQL 10.0.2, Microsoft.EntityFrameworkCore.SqlServer 10.0.4
Dapper	Dapper 2.1.79, Npgsql 10.0.3, Microsoft.Data.SqlClient 6.0.2
Broker	RabbitMQ.Client 7.x (async API), Aspire.RabbitMQ.Client 13.4.x
Aspire	Aspire.Hosting.* 13.4.x (Postgres / SqlServer / RabbitMQ)
Tests	xUnit v3 3.2.2, Shouldly, NSubstitute, Testcontainers 4.12.x, BenchmarkDotNet 0.14

2. Architecture and layers

The project follows a clean, layered architecture that separates **commands** from **queries**. The key rule is that dependencies point inward, toward the domain:



Project	Depends on	Responsibility
<code>DataAccess.Domain</code>	—	Aggregates (Order, Product), value objects (Money), domain events, invariants. No reference to EF or infrastructure.
<code>DataAccess.Application</code>	Domain	Contracts (repositories, <code>IUnitOfWork</code> , <code>IOrderQueries</code>), commands+handlers, <code>Result</code> , DTOs, keyset paging.
<code>Infrastructure.EfCore</code>	Domain, Application	Command side: <code>DbContext</code> , configurations, repositories, Outbox/Inbox, migrations, DI.
<code>Infrastructure.Dapper</code>	Application	Query side: hand-written SQL, dialects, connection factory, multi-mapping.
<code>DataAccess.Api</code>	all of the above	Minimal API, DI wiring, Outbox/consumer hosted services, demo endpoints.
<code>AppHost / ServiceDefaults</code>	—	Aspire orchestration of resources; shared telemetry/health-check defaults.

Feature-first. Folders are named by feature/aggregate (`Orders/` , `Products/`), not by technical type (`Repositories/` , `Services/`). The namespace always matches the folder; namespaces are file-scoped.

3. Domain building blocks (DDD)

Entity — identity that persists

An entity equals another when it is the same type and the same `Id`, regardless of other fields. The base class captures this:

```
public abstract class Entity<TId> where TId : notnull
{
    public TId Id { get; protected set; } = default!;
    public override bool Equals(object? obj) => obj is Entity<TId> other
        && GetType() == other.GetType()
        && EqualityComparer<TId>.Default.Equals(Id, other.Id);
    public override int GetHashCode() => GetHashCode.Combine(GetType(), Id);
}
```

The private parameterless constructor is needed by EF Core to materialize the object from the database (EF creates the instance and fills fields, bypassing the public API).

ValueObject — an object with no identity

Equal to another when all "equality components" match. A subtype lists them in `GetEqualityComponents()`:

```
public abstract class ValueObject
{
    protected abstract IEnumerable<object?> GetEqualityComponents();
    public override bool Equals(object? obj) => obj is ValueObject other
        && GetType() == other.GetType()
        && GetEqualityComponents().SequenceEqual(other.GetEqualityComponents());
    // GetHashCode aggregates the components via GetHashCode.Add( ... )
}
```

We deliberately do **not** overload operator `==` on reference-type value objects (SonarQube rule S3875): overriding `Equals` / `GetHashCode` is enough, and operators on reference types mislead the reader.

Money — a value object example

`Money` = an amount plus an ISO currency code. It is immutable, compared by value, and rejects operations across different currencies and negative amounts (otherwise `DomainException`). The currency is normalized to upper case.

AggregateRoot — the aggregate root

An aggregate is the transaction boundary and the single entry point. Only the root records domain events:

```

public abstract class AggregateRoot<TId> : Entity<TId>, IHasDomainEvents
{
    private readonly List<IDomainEvent> _domainEvents = [];
    public IReadOnlyList<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();
    protected void RaiseDomainEvent(IDomainEvent e) => _domainEvents.Add(e);
    public void ClearDomainEvents() => _domainEvents.Clear();
}

```

`IHasDomainEvents` is a **non-generic** interface. Infrastructure (the Outbox interceptor) uses it to walk the `ChangeTracker` and collect events from every aggregate without knowing the concrete `TId`.

Invariants: Order and Product

Aggregate	Invariants
Order	Lines change only while in <code>Draft</code> ; an order cannot be placed empty; all lines share one currency; <code>Place()</code> raises <code>OrderPlaced</code> ; <code>MarkPaid()</code> only from <code>Placed</code> ; <code>Cancel()</code> not from <code>Paid</code> / <code>Cancelled</code> .
Product	<code>Stock</code> is never negative; <code>Decrease()</code> to zero raises <code>StockDepleted</code> .

DomainException vs Result

Split by the nature of the outcome:

- **Invariant violation** (should not happen with correct code) → `DomainException` (an exception).
- **Expected business outcome** (e.g. "not enough stock") → `Result` at the application layer, without throwing.

4. Strongly-typed IDs and auto-converters

The problem. If every identifier is just a `Guid`, it is easy to swap `OrderId` and `ProductId` in arguments and the compiler will not help. The fix is a dedicated type per ID:

```
public readonly record struct OrderId(Guid Value) : IStronglyTypedId
{
    public static OrderId New() => new(Guid.NewGuid());
    public override string ToString() => Value.ToString();
}
```

`readonly record struct` gives value equality, immutability and no heap allocation for free.

One problem — one generic converter

EF does not know how to store an `OrderId`. Writing a `ValueConverter` per ID is the naive path. Instead we use a **single** generic converter plus a marker interface:

```
public interface IStronglyTypedId { Guid Value { get; } }

public sealed class StronglyTypedIdConverter<TId>()
    : ValueConverter<TId, Guid>(id => id.Value, FromGuid())
    where TId : struct, IStronglyTypedId
{
    private static Expression<Func<Guid, TId>> FromGuid()
    {
        var ctor = typeof(TId).GetConstructor([typeof(Guid)])!;
        var value = Expression.Parameter(typeof(Guid), "value");
        return Expression.Lambda<Func<Guid, TId>>(Expression.New(ctor, value), value);
    }
}
```

The forward direction (`id => id.Value`) is trivial. The reverse is built with an **expression tree**: find the `TId(Guid)` constructor and assemble the lambda `value => new TId(value)` — because the concrete type is unknown when the converter is compiled.

Auto-registration via reflection

`AppDbContext.ConfigureConventions` scans the domain assembly once and applies the converter to *every* type implementing the marker:

```

private static readonly Type[] StronglyTypedIdTypes = typeof(IStronglyTypedId).Assembly
    .GetTypes()
    .Where(t => t is { IsValueType: true } && typeof(IStronglyTypedId).IsAssignableFrom(t))
    .ToArray();

protected override void ConfigureConventions(ModelConfigurationBuilder b)
{
    foreach (var idType in StronglyTypedIdTypes)
        b.Properties(idType).HaveConversion(
            typeof(StronglyTypedIdConverter<>).MakeGenericType(idType));
}

```

Result: a new ID (say `InvoiceId`) only needs to declare `: IStronglyTypedId` — EF picks it up automatically, with no new mapping code. `MakeGenericType` substitutes the concrete type into the open generic `StronglyTypedIdConverter<>` at runtime.

5. EF Core: the command (write) side

An abstract DbContext + provider-specific subclasses

```
public abstract class AppDbContext(DbContextOptions options) : DbContext(options), IUnitOfWork
{
    public DbSet<Order> Orders => Set<Order>();
    public DbSet<Product> Products => Set<Product>();
    public DbSet<OutboxMessage> OutboxMessages => Set<OutboxMessage>();
    public DbSet<InboxMessage> InboxMessages => Set<InboxMessage>();

    protected override void OnModelCreating(ModelBuilder b) =>
        b.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
}
```

Why `DbSet<T> => Set<T>()` instead of `{ get; set; }`?

Both work, but the expression-bodied property is better: it is *computed* (no setter that nobody should call), does not rely on EF assigning a value, and works cleanly in an abstract base class. `Set<T>()` returns the same cached `DbSet` instance.

`ApplyConfigurationsFromAssembly`

Scans the assembly and applies every `IEntityTypeConfiguration<T>`. This keeps the mapping next to the entity (one file per aggregate) and stops `OnModelCreating` from bloating.

Subclasses are thin and only add provider differences:

```
// Postgres
b.Entity<Product>().Property<uint>("xmin").HasColumnName("xmin").IsRowVersion();
b.Entity<OutboxMessage>().Property(m => m.Content).HasColumnType("jsonb");
// SQL Server
b.Entity<Product>().Property<byte[]>("RowVersion").IsRowVersion();
b.Entity<OutboxMessage>().Property(m => m.Content).HasColumnType("nvarchar(max)");
```

IUnitOfWork — dependency inversion

`AppDbContext` implements `IUnitOfWork` (the `SaveChangesAsync` signature matches). Handlers in Application depend on the `IUnitOfWork` interface, not on EF — so Domain and Application stay free of any EF reference. A "unit of work" is one command's commit transaction.

6. Owned types: OwnsOne / OwnsMany

Owned types are entities without their own identity that EF stores as part of an owner. Here that is `Money` (inside `Product` and order lines) and the collection of order lines.

Technique	What it does
<code>OwnsOne</code>	One owned object in the owner's own table (columns <code>Price_Amount</code> , <code>Price_Currency</code>).
<code>OwnsMany</code>	A collection of owned objects in a <i>separate</i> table with an FK to the owner (<code>order_lines</code>).
<code>HasMany</code> / <code>HasOne</code>	A relationship to <i>independent</i> entities (own lifecycle and identity) — not our case for lines.
<code>WithOwner</code>	The inverse side of ownership (FK from owned to owner) inside <code>OwnsMany</code> .
<code>Navigation(...)</code>	Fine-tuning a navigation (e.g. <code>IsRequired()</code> or the access mode).

```
builder.OwnsMany(o => o.Lines, lines =>
{
    lines.ToTable("order_lines");
    lines.HasKey(l => l.Id);
    lines.OwnsOne(l => l.UnitPrice, price => price.ConfigureMoney()); // shared helper
    lines.Navigation(l => l.UnitPrice).IsRequired();
    lines.Ignore(l => l.LineTotal);           // computed property – do not store
});
builder.Navigation(o => o.Lines).UsePropertyAccessMode(PropertyAccessMode.Field);
builder.Ignore(o => o.Total);                // computed from lines
builder.Ignore(o => o.DomainEvents);         // not persistent state
```

Why `PropertyAccessMode.Field` ?

`Order.Lines` is a read-only projection (`_lines.AsReadOnly()`). The property has no setter, so EF must read/write the backing field `_lines` directly rather than through the property. This preserves encapsulation: from the outside the collection is immutable and can only change through aggregate methods (`AddLine` / `RemoveLine`).

Why `Ignore` ?

`Total` and `LineTotal` are computed (not stored state), and `DomainEvents` is the aggregate's bookkeeping collection. None of the three belongs in the schema.

An easy thing to break. Inside `OwnsOne(Money)` the lines `price.Property(m => m.Amount)` and `price.Property(m => m.Currency)` are **load-bearing**, not "noise". `Money` exposes get-only properties, and EF needs these calls to bind the columns to constructor parameters during materialization. Remove them and the model fails with "No suitable constructor for Money". The `ModelBuildsTests` test catches this.

Shared Money mapping

So that `Money` maps the same way everywhere (and `numeric(18,2)` in PG matches `decimal(18,2)` in SQL Server), the mapping lives in one extension:

```
internal static void ConfigureMoney<TOwner>(this OwnedNavigationBuilder<TOwner, Money> money)
    where TOwner : class
{
    money.Property(m => m.Amount).HasPrecision(18, 2);
    money.Property(m => m.Currency).HasMaxLength(3);
}
```

Status as `int`, not a string

`builder.Property(o => o.Status).HasConversion<int>()` stores the enum as a number. It is more compact and index-friendly; string storage reads better in the database but is sensitive to renaming values. For an internal status, `int` was chosen.

7. Optimistic concurrency: xmin and rowversion

The lost-update problem. Two processes read a product with `Stock = 10`, each decrements by 1 and saves. With no protection the second silently overwrites the first, and the database ends up with 9 instead of 8.

The optimistic strategy: do not lock up front; at save time verify that nobody changed the row since it was read. The mechanism is a *version token* that changes on every update. EF appends it to the `WHERE` :

```
UPDATE products SET "Stock" = 9 WHERE "Id" = @id AND <version> = @versionRead;
-- 0 rows affected => someone got there first => DbUpdateConcurrencyException
```

PostgreSQL: the system column `xmin`

In Postgres every row has a hidden system column `xmin` — the id of the transaction that created the current version of the row. Any `UPDATE` physically rewrites the row as a new version with a new `xmin`. That makes `xmin` a ready-made, "free" concurrency token: no extra schema column needed.

```
modelBuilder.Entity<Product>()
    .Property<uint>("xmin")           // shadow property (no field in the C# class)
    .HasColumnName("xmin")
    .IsRowVersion();
```

Npgsql 10: the former helper `UseXminAsConcurrencyToken()` has been removed. The current approach is to declare a shadow `uint "xmin"` property and mark it `IsRowVersion()`, as above.

SQL Server: `rowversion`

SQL Server has no `xmin`, but it has the `rowversion` type (a.k.a. `timestamp`) — an 8-byte value the engine bumps on every change to a row within the database. We store it as a shadow property:

```
modelBuilder.Entity<Product>().Property<byte[]>("RowVersion").IsRowVersion();
```

Shadow properties

Both `xmin` and `RowVersion` are **shadow properties**: present in the EF model and the database, but *absent* from the C# `Product` class. The domain should not know about an infrastructure token — which is why it does not appear in domain code (the single comment in `Product` notes this).

Demo and test

The `POST /demo/concurrency` endpoint and the

`Concurrent_stock_decrement_is_detected_as_a_conflict` test open two independent scopes/contexts, both read the product (one token), both decrement, both save. The first commit succeeds, the second catches `DbUpdateConcurrencyException` — the lost update is detected, not silent.

8. Pessimistic locking and isolation

The optimistic strategy is great when conflicts are *rare*. When contention is high (and retrying is expensive), use **pessimistic** locking: lock the row up front so others wait.

	Optimistic	Pessimistic
When	conflicts are rare	conflicts are frequent
Mechanism	version token in <code>WHERE</code>	row lock inside a transaction
On conflict	the loser gets an exception	the second waits for the first (serialized)
Cost	retries	waiting / deadlock risk

Locking is expressed differently per provider (a dialect in the Dapper layer):

```
-- PostgreSQL: row-level lock
SELECT "Stock" FROM products WHERE "Id" = @id FOR UPDATE;
-- SQL Server: table hints
SELECT Stock FROM products WITH (UPDLOCK, ROWLOCK) WHERE Id = @id;
```

The `POST /demo/pessimistic` endpoint runs *two* competing updates. Each opens its own transaction, locks the row, reads stock, decrements and commits. The second writer blocks on `FOR UPDATE` until the first commits — the decrements serialize and nothing is lost. The returned "stock read under lock" values differ, which proves the serialization.

Isolation levels

The `GET /demo/isolation?level=Serializable` endpoint shows controlling the transaction isolation level via `BeginTransactionAsync(IsolationLevel, ct)`.

Level	Prevents
ReadCommitted (default)	dirty reads
RepeatableRead	+ non-repeatable reads
Serializable	+ phantoms (full serializability, most expensive)

EF retries (`EnableRetryOnFailure`) conflict with *manual* transactions (`BeginTransaction`): the retrying execution strategy forbids them. That is why retries are off by default here (see §15).

9. Migrations and design-time factories

Each provider has its own set of migrations (`Migrations/Postgres` , `Migrations/SqlServer`), generated from the corresponding context.

Design-time factories

`dotnet ef` must be able to build a context outside the runtime. `IDesignTimeDbContextFactory<T>` handles this: the connection string comes strictly from an environment variable, and if it is missing the factory **fails fast** with an exception (no "magic" localhost defaults):

```
var cs = Environment.GetEnvironmentVariable($"ConnectionStrings__{DatabaseConnectionNames.Postgres}");
?? throw new InvalidOperationException("Connection string ... is not set.");
```

Resource names (`orders-postgres` / `orders-sqlserver`) live in a single `DatabaseConnectionNames` constant, kept in sync with `AppHost` .

Practical notes

- `.editorconfig`: the section `[**/Migrations/**/*.cs]` `generated_code = true` — generated code should not obey our style rules (otherwise `IDE0161` breaks the build under `TreatWarningsAsErrors`).
- `Money.Amount` uses `HasPrecision(18,2)` on both providers, so `numeric(18,2)` and `decimal(18,2)` line up.
- Check without a database: `dotnet ef migrations has-pending-model-changes` for both contexts.
- `migrations add` does *not* connect to the database — a format-valid connection string in the env is enough.
- The `inbox_messages` table (§14) requires its own migration for both providers — commands in §22.

10. Dapper: the query (read) side

For reads-for-display, aggregates are overkill: no tracking, invariants or whole-graph loading needed. Dapper runs hand-written SQL and maps the result into **flat DTOs**.

Connection factory and dialects

`ISqlConnectionFactory` creates a `DbConnection` for the chosen provider. `OrderDialect` encapsulates SQL differences: identifier quoting, the keyset predicate, and the row-limit clause.

Multi-mapping: order + lines in one query

A single `JOIN` returns "wide" rows (the order header repeats for each line). Dapper splits each row into two objects at the `splitOn` marker:

```
await connection.QueryAsync<OrderHeaderRow, OrderLineRow?, OrderHeaderRow>(
    command,
    (order, line) =>           // called for EVERY result row
    {
        header ??= order;      // take the header once
        if (line is not null) lines.Add(line);
        return order;
    },
    splitOn: "LineId");
```

How `splitOn` works

Dapper walks the columns left to right and, when it reaches a column named `LineId`, starts mapping the *second* type (`OrderLineRow`). Everything before it is the first type (`OrderHeaderRow`). That is why the SQL names the line column `l."Id" AS LineId` — it is the split point. The generic parameters `<THeader, TLine, TReturn>` define the piece types and the type the lambda returns (the return value does not matter here — we accumulate state in the outer `header / lines`).

`OrderLineRow?` is nullable because a `LEFT JOIN` on an order with no lines yields `NULL` in the line columns.

11. Keyset pagination

The OFFSET problem. `OFFSET 10000 LIMIT 20` forces the database to read and discard 10,000 rows — slower the further you page. And with inserts/deletes the pages "drift".

Keyset (seek). Instead of a page number, the cursor is the *sort-key values of the last row seen*. The next page = "rows that come **after** the cursor". We sort by `(CreatedAt, Id)` — the second key guarantees a stable order when dates tie.

```
-- Postgres: row-value comparison
WHERE ("CreatedAt", "Id") > (@afterCreatedAt, @afterId)
ORDER BY "CreatedAt", "Id" LIMIT @limit;

-- SQL Server: no row-value comparison ⇒ expand into OR
WHERE (CreatedAt > @afterCreatedAt
      OR (CreatedAt = @afterCreatedAt AND Id > @afterId))
ORDER BY CreatedAt, Id OFFSET 0 ROWS FETCH NEXT @limit ROWS ONLY;
```

The "N+1 rows" trick

To learn whether there is a next page, fetch `PageSize + 1` rows. If more than `PageSize` came back, the extra "tail" row signals a next page; drop it, and take the cursor from the last *shown* row:

```
public static KeysetPage<T> Build<T>(IReadOnlyList<T> rows, int pageSize,
    Func<T, DateTimeOffset> createdAt, Func<T, Guid> id)
{
    var hasMore = rows.Count > pageSize;
    var items = (hasMore ? rows.Take(pageSize) : rows).ToArray();
    if (!hasMore || items.Length == 0)
        return new KeysetPage<T>(items, null, null, hasMore);
    var last = items[^1];
    return new KeysetPage<T>(items, createdAt(last), id(last), true);
}
```

`KeysetPageBuilder` is a pure function with no database, so it is easy to unit-test (edge cases: empty page, exactly `PageSize`, `PageSize+1`).

Keyset only truly "seeks along the index" with an index on `(CreatedAt, Id)`. For honest seek pagination in production, that index is required.

12. CQRS and its levels

CQRS (Command Query Responsibility Segregation) separates *changing* from *reading*. Here it is literally visible: commands go through EF (aggregates), queries through Dapper (flat DTOs).

Level	Idea	In this project
1. Separate models	Different read/write models over one database	✔ EF aggregates + Dapper DTOs over the same tables
2. Separate read model	Materialized <code>VIEW</code> s / denormalization for reads	Described conceptually; inline SQL was chosen for clarity of JOIN/aggregation/dialects
3. Separate store	A separate read database updated by projections from events	The foundation exists (Outbox events) — the natural next step

Why level 2 (a `VIEW`) was not built: for a learning project, inline SQL is clearer — the JOIN, aggregation and dialect differences are visible in code, whereas a `VIEW` would hide them in the schema.

13. The transactional Outbox

The dual-write problem. When an order is placed you must both write it to the database and publish an event to RabbitMQ. These are two different resources — they cannot be committed atomically. Crash between them and either the order exists but the event is lost, or the event is sent but the order was not saved (a phantom).

The Outbox solution. The event is written to the `outbox_messages` table *in the same transaction* as the state change. The transaction is atomic, so the event is saved if and only if the state was saved. A separate background process later reads the outbox and publishes to the broker.

OutboxInterceptor — capturing events in the transaction

A `SaveChangesInterceptor` hooks `SaveChangesAsync`: it walks the `ChangeTracker`, collects domain events from every aggregate, serializes them into `OutboxMessage` rows added to the same context (hence the same transaction), then clears the aggregate's events.

```
private void CaptureDomainEvents(DbContext context)
{
    var occurredAt = timeProvider.GetUtcNow(); // single timestamp, see §16
    foreach (var aggregate in context.ChangeTracker
        .Entries<IHasDomainEvents>().Select(e => e.Entity)
        .Where(a => a.DomainEvents.Count > 0))
    {
        foreach (var e in aggregate.DomainEvents)
            context.Add(new OutboxMessage {
                Id = Guid.NewGuid(),
                Type = e.GetType().Name,
                Content = JsonSerializer.Serialize(e, e.GetType()),
                OccurredAt = occurredAt,
            });
        aggregate.ClearDomainEvents();
    }
}
```

Time is stamped here, not in the domain. Domain code never reads the clock — this keeps the "no `UtcNow` in the domain" rule and makes time deterministic in tests. A single timestamp is set by the interceptor through `TimeProvider` (§16).

OutboxProcessor — the relay

A hosted service periodically (every 2 s, batch of 20) selects `WHERE ProcessedAt IS NULL ORDER BY OccurredAt`, publishes `Content` to the queue, and sets `ProcessedAt`. It publishes with a `MessageId` and `Persistent = true`. This is **at-least-once** delivery: crash after publishing but before `SaveChanges` and the message is sent again — so the consumer must be idempotent (§14).

Storing Content : jsonb in Postgres (indexable, validated JSON) vs nvarchar(max) in SQL Server.

14. Inbox and the idempotent consumer

Because delivery is at-least-once, the consumer will see duplicates. **Idempotency** means processing a repeated message has no additional effect. This is the "second half" of the pattern: Outbox + Inbox yield an *exactly-once* effect on top of an at-least-once transport.

The `inbox_messages` table

Processed `MessageId`s are stored in the database, and the **primary key** provides idempotency even when two deliveries race:

```
private async Task<bool> TryRecordAsync(string messageId, CancellationToken ct)
{
    using var scope = scopeFactory.CreateScope();
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    if (await db.InboxMessages.AnyAsync(m => m.MessageId == messageId, ct))
        return false; // already processed
    db.InboxMessages.Add(new InboxMessage { MessageId = messageId, ProcessedAt = ... });
    try { await db.SaveChangesAsync(ct); return true; }
    catch (DbUpdateException) { return false; } // race: PK conflict => duplicate
}
```

The `AnyAsync` check filters out most repeats; the `try/catch (DbUpdateException)` closes the race window when two deliveries both pass the check — only one insert wins, the other hits a PK conflict and is recognized as a duplicate.

The integration test `Duplicate_delivery_is_processed_once` publishes one message with a fixed `MessageId` twice and asserts that `inbox_messages` holds exactly one row.

RabbitMQ details

- Queue topology is declared in one place (`OrderEventsTopology.DeclareAsync`): a durable queue, not exclusive, not auto-delete.
- `AsyncEventingBasicConsumer` + manual `BasicAckAsync` (`autoAck: false`) — acknowledge only after processing.
- `BasicProperties.MessageId` = the outbox Id (end-to-end correlation), `Type` = the event name.

15. Modern EF Core techniques

The `/ef/*` endpoints show queries beyond "load-mutate-save".

Technique	What and why
<code>ExecuteUpdateAsync</code>	A bulk <code>UPDATE</code> in one statement, with no entities materialized or tracked.
<code>ExecuteDeleteAsync</code>	A bulk <code>DELETE</code> (e.g. purging already-relayed outbox rows).
<code>FromSql</code> + LINQ	Raw SQL as the source, with EF translating <code>Where/OrderBy</code> on top — the raw part stays provider-neutral.
<code>AsSplitQuery</code>	Loads the owner and its collection as separate queries instead of one JOIN — against the "cartesian explosion" of rows.
<code>AsNoTracking</code>	Reads without change-tracker snapshots — faster and lighter on memory.

Important: `ExecuteUpdate/ExecuteDelete` bypass `SaveChanges` and therefore do **not** go through the Outbox interceptor — they are for maintenance, not domain mutations.

Cartesian explosion

Loading an owner with several collections in one JOIN multiplies the rows ($N \text{ rows} \times M \text{ rows} = N \cdot M$ duplicated headers), and the result set blows up. `AsSplitQuery` breaks it into several simple queries.

EnableRetryOnFailure

Transient-fault retries are enabled by the `Database:EnableRetryOnFailure` flag (off by default). The caution: the retrying execution strategy forbids *user-initiated* transactions (`BeginTransaction`), which the concurrency/isolation demos and integration tests rely on. The correct way to combine them is to wrap an explicit transaction in `db.Database.CreateExecutionStrategy().ExecuteAsync( ... )`.

16. TimeProvider and deterministic time

`TimeProvider` (introduced in .NET 8) is an abstraction over the clock. Instead of calling `DateTime.UtcNow` / `DateTimeOffset.UtcNow` directly, code takes time from the provider, which makes it **testable**: a test can substitute a fixed or controllable time.

- Registration: `services.AddSingleton(TimeProvider.System)`.
- Handlers receive a `TimeProvider` and pass time into the domain (`Order.CreateDraft(customerId, timeProvider.GetUtcNow())`).
- The `OccurredAt` of outbox messages is stamped by the interceptor via `TimeProvider` — a single point; the domain never reads the clock.
- `OutboxProcessor` sets `ProcessedAt` through `TimeProvider` too.

To honor this rule, domain events carry no `OccurredAt` parameter; the domain is fully free of clock access.

17. Result vs exceptions; HTTP mapping

`Result` / `Result<T>` is an explicit outcome type for *expected* business situations (success with a value, or failure with a message). `Result` 's own invariants are guarded: a success cannot carry an error, and a failure must carry one.

```
public static Result<T> Success<T>(T value) ⇒ new(value, true, null);
public static Result<T> Failure<T>(string error) ⇒ new(default, false, error);
public T Value ⇒ IsSuccess ? _value! : throw new InvalidOperationException( ... );
```

Extension methods translate `Result` into HTTP: a failure is an expected business outcome, so it maps to 400 rather than an exception:

```
ToCreated(loc) → 201 Created | 400 BadRequest{error}
ToOk()          → 200 OK      | 400 BadRequest{error}
ToNoContent()   → 204         | 400 BadRequest{error}
```

A command and its handler live in one file (`XxxCommand` + `XxxCommandHandler`), without MediatR; handler logic is split into private methods (e.g. `PlaceOrderCommandHandler : LoadProducts → ValidateItems → AssembleOrder` , where validation fully precedes mutation so a rejected order leaves no side effects).

18. Aspire, AppHost and ServiceDefaults

.NET Aspire is a model for locally orchestrating a distributed application. `AppHost` declaratively brings up resources and wires them to services:

```
var postgres = builder.AddPostgres("orders-postgres").AddDatabase("ordersdb");
var sqlserver = builder.AddSqlServer("orders-sqlserver").AddDatabase("OrdersDb");
var rabbit = builder.AddRabbitMQ("rabbitmq").WithManagementPlugin();
builder.AddProject<Projects.DataAccess_Api>("api")
    .WithReference(postgres).WithReference(rabbit).WaitFor(postgres);
```

- `WithReference` injects connection strings into the service via configuration (`ConnectionStrings__ ...`).
- `WaitFor` delays the Api's start until the resource is ready.
- `ServiceDefaults` (`AddServiceDefaults`) adds OpenTelemetry, health checks, service discovery and HTTP resilience — a shared "baseline" for every service.
- Client integrations (`AddRabbitMQClient("rabbitmq")`) configure the client from the connection string Aspire supplies.

At startup the Api applies the active provider's migrations (`MigrateAsync`) — convenient for dev/demo; in production migrations are usually moved to a separate deployment step.

19. Testing

Three levels, each with its own role:

Level	What it checks	Tools
Unit	Domain (Money, Order, Product) and Application handlers — fast, no database	xUnit v3, Shouldly, NSubstitute
Model builds	Consistency of both providers' EF models without a database connection	xUnit + <code>DbContextOptionsBuilder</code>
Integration	Real queries against real databases/broker through the real DI	Testcontainers (Postgres/MsSql/RabbitMq)

"Model builds" — a cheap but powerful check

The test builds `context.Model` without opening a connection. If the mapping is inconsistent (e.g. a missing `Property` on owned `Money`), the failure happens here — fast and without a database. This is the test that catches the regression from §6.

Testcontainers — "production-like" integration

Fixtures spin up real containers and drive them through the *same* DI (`AddEfCore` / `AddDapper` + real handlers) plus `MigrateAsync`. So the tests exercise the production wiring, not a bespoke setup. `TestData` seeds data through the real command handlers.

What is covered: EF-write → Dapper-read round-trip, keyset pagination with no duplicates/gaps, optimistic concurrency (xmin/rowversion), transaction rollback, outbox capture, reading under Serializable, end-to-end Outbox (Postgres+RabbitMQ with the real `OutboxProcessor`), consumer idempotency (Inbox), and a SQL Server subset.

xUnit v3 notes

- `IAsyncLifetime` now returns `ValueTask`.
- A container shared across a set — via `ICollectionFixture` + `[Collection]`; per-class — `IClassFixture`.
- Tests pass `TestContext.Current.CancellationToken` (otherwise the analyzer complains — xUnit1051).

Compilation and test discovery are green without Docker; a **green run** of the integration tests requires a running Docker runtime.

20. EF vs Dapper benchmark

The `.DataAccess.Benchmarks` project (BenchmarkDotNet) measures reading one order-with-lines three ways against the *same* database, so the numbers reflect mapping cost rather than different databases:

Method	What it does
Dapper (baseline)	Hand-written multi-mapping into flat row types.
EfCore_Tracking	Loads the aggregate with tracking (a snapshot of every entity in the tracker).
EfCore_NoTracking	The same with <code>AsNoTracking()</code> — no snapshots.

- `[MemoryDiagnoser]` — reports allocations alongside time.
- `[GlobalSetup]` starts one Postgres container, migrates and seeds; `[GlobalCleanup]` tears it down.
- An in-process toolchain keeps all benchmarks in one process, so there is **one** container rather than one per method.

The expected picture: Dapper is usually faster and lighter on memory for pure reads; EF with tracking is the most expensive (it pays for the identity map and snapshots), no-tracking sits in between. This clearly justifies choosing Dapper on the query side.

```
Run: dotnet run -c Release --project benchmarks/DataAccess.Benchmarks (BenchmarkDotNet
requires Release, and Docker for the container).
```

21. Conventions and code quality

- **TreatWarningsAsErrors=true** (except NU1902/NU1903 — advisory vulnerabilities do not break the build), `EnforceCodeStyleInBuild=true` .
- **Nullable reference types** enabled; for required strings we skip `.IsRequired()` — NRT already makes the column NOT NULL.
- **Comments explain only "why"**. Ceremonial XML-doc that restates the type name is avoided; explanations of non-obvious decisions remain (xmin, keyset +1, CQRS boundaries, Outbox atomicity, dialect differences).
- File-scoped namespaces; primary constructors; `sealed` by default.
- Configuration without noise: do not write what EF's conventions already do (`HasKey` on `Id` , a bare `Property(x)`).
- No Central Package Management (versions in `.csproj`); the solution is `.slnx` .

22. How to run

Orchestration (Aspire)

```
dotnet run --project src/DataAccess.AppHost # brings up Postgres, SQL Server, RabbitMQ and the A
```

Migrations

The connection string comes from the env (resource name contains a hyphen). Example for Postgres:

```
$env:ConnectionStrings__orders-postgres = "Host=localhost;Database=ordersdb;Username=postgres;Pass
dotnet ef migrations add AddInboxMessages `
  --project src/DataAccess.Infrastructure.EfCore `
  --startup-project src/DataAccess.Infrastructure.EfCore `
  --context PostgresAppDbContext --output-dir Migrations/Postgres
# likewise for SqlServerAppDbContext → Migrations/SqlServer (env ... __orders-sqlserver)
```

Tests and benchmark

```
dotnet test # unit + integration (integration needs D
dotnet run -c Release --project benchmarks/DataAccess.Benchmarks
```

Main endpoints

Method/path	Purpose
POST /products , POST /products/{id}/restock	Create a product / restock (commands, EF)
POST /orders , GET /orders/{id} , GET /orders	Place an order; details (Dapper multi-map); list (keyset)
GET /outbox	Inspect outbox messages
POST /demo/concurrency , POST /demo/pessimistic , GET /demo/isolation	Optimistic/pessimistic locking, isolation levels
/ef/*	ExecuteUpdate/Delete, FromSql, AsSplitQuery

23. Glossary

Term	In short
Aggregate / aggregate root	A cluster of objects with one consistency boundary; the root is the single entry point and the transaction boundary.
Value object	An object with no identity, equal by value (Money).
Strongly-typed ID	A dedicated wrapper type over Guid so identifiers cannot be confused.
Shadow property	A property in the EF model/database that is absent from the C# class (xmin, RowVersion).
Owned type	An entity without its own identity, stored as part of its owner.
Optimistic locking	Checking a version token at save time; a conflict → an exception.
Pessimistic locking	An explicit row lock inside a transaction (FOR UPDATE / UPDLOCK).
xmin	A Postgres system column — the id of the transaction that created the row version; a ready-made concurrency token.
rowversion	An 8-byte auto-incremented row-version token in SQL Server.
CQRS	Separating the responsibility of commands (writes) and queries (reads).
Outbox	A table where an event is written in the same transaction as the state; later relayed to the broker.
Inbox	A table of processed Messages for consumer idempotency.
At-least-once	An "at least once" guarantee: duplicates are possible, so idempotency is required.
Keyset pagination	Paging by a cursor (sort-key values) rather than OFFSET.
Multi-mapping	Splitting one result row into several objects (Dapper splitOn).
Unit of Work	The boundary of one transaction; implemented here by the DbContext.
TimeProvider	A clock abstraction for testable time.